Sarah Benavides

CS 362 Lab 7

3/18/2026

# Windows Forensics Part II

Part 1: Non-technical summary

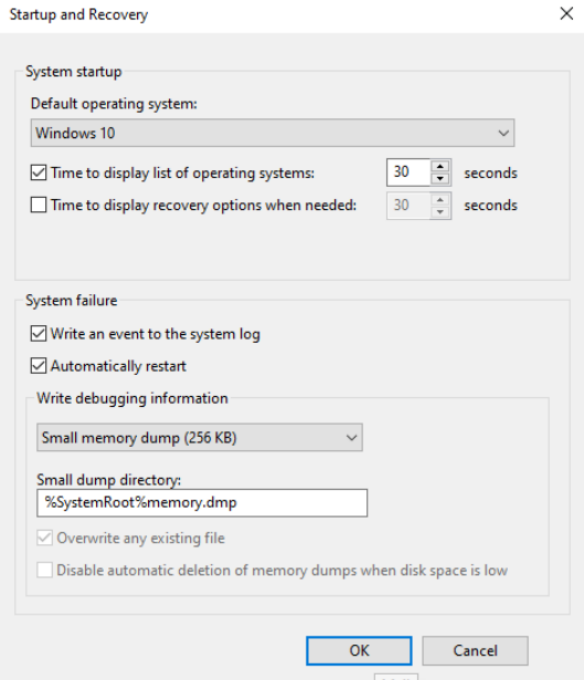
In this lab, we experimented with steps three and four of the Windows Forensics methodology: Windows memory analysis and Windows registry analysis. We were introduced to several techniques that are used to collect and examine information within a Windows system. In the first part, we focused on creating and analyzing Windows crash dump files. These files are generated when the system fails and capture the computer's memory at the time of the crash. This information can be used to potentially identify why the system failed.

We also examined the information that belongs to the processes running on the system. By changing a command slightly that is used to list process information, we were able to focus on individual active programs instead of analyzing the whole memory. Identifying running processes and their process IDs allows us to gather additional information about specific programs and capture data for further examination.

The last part of this lab touched on live RAM acquisition. We were given the option of using two different applications to create a memory capture. When using this application, we were able to collect the contents of RAM while the system was still running.

Part 2: Technical summary

In this lab, we focused on three major sections within Windows forensics: Windows crash dumping, collecting process information, and RAM acquisition. The first part walks us through a crash dump. When a system fails, a crash dump is the storage space where the system stores a memory backup. A crash dump is used to identify the reasons for a system failure and can be used to collect information about the system state, memory locations, application and program status, etc. To create a memory dump, we start by using our Windows virtual machine to access the Startup and Recovery window. After opening the virtual machine, we follow these steps in order: **SYSTEM > ABOUT > ADVANCED SYSTEM SETTINGS > ADVANCED > STARTUP AND RECOVER > SETTINGS [>WRITE DEBUGGING INFORMATION]** and choose small memory dump (256 KB).




When I tried to run the command **dir *.dmp**, an error appeared stating that the file was not found. I went back to Lab 6 to create a new memory dump to see if this was the issue. I changed the directory by entering **cd C:\Users\vboxuser\Documents\SysinternalsSuite**, then ran **procexp** to open the Process Explorer window. I right-clicked on the process **msedge.exe** and selected **Create Dump** and **Create Mini Dump**. I then changed the directory by inputting cd C:\Windows\ to check if the **.dmp** file was present, and it was.

```
C:\Windows>cd C:\Users\vboxuser\Documents\SysinternalsSuite

C:\Users\vboxuser\Documents\SysinternalsSuite>procexp

C:\Users\vboxuser\Documents\SysinternalsSuite>cd C:\Windows\

C:\Windows>dir *.dmp
 Volume in drive C has no label.
 Volume Serial Number is 4612-7786

 Directory of C:\Windows

02/25/2026  04:34 PM         9,635,519 msedge.dmp
               1 File(s)      9,635,519 bytes
               0 Dir(s)  27,040,632,832 bytes free
```

In order to use the command **dumpchk**, I input **winget install Microsoft.WinDbg** to install this tool and ultimately analyze the crash dump files. When attempting to change the directory to **C:\Program Files (x86)\Windows Kits\10\Debuggers\x64**, an error appeared stating no path was found. I realized I needed to install Windows SDK and did so by inputting the command **winget install –source winget –exact –id Microsoft.WindowsSDK.10.0.26100**. Once this was installed, I changed the directory and input the command **dumpchk C:\Windows\msedge.DMP | findstr "Bug*"**. This file only contained a build number, which is shown in the screenshot below.

```
C:\Program Files (x86)\Windows Kits\10\Debuggers\x64>dumpchk C:\Windows\msedge.DMP | fi
ndstr "Bug*"
Loading dump file C:\Windows\msedge.DMP
  BuildNumber              00004A65 (19045)
```

In the second part of this lab, we concentrate on collecting process information. Sometimes investigators want to solely analyze a single service or maybe a group of services instead of the whole memory. To access the tool pslist, I changed the directory to **C:\Users\vboxuser\Documents\SysinternalsSuite**. After changing the directory, I input the command **pslist -nobanner** to display a list of processes divided into different categories within the device.

```
C:\Windows\system32>cd C:\Users\vboxuser\Documents\SysinternalsSuite

C:\Users\vboxuser\Documents\SysinternalsSuite>pslist -nobanner
Process information for WINDOWS:

Name                Pid Pri Thd  Hnd   Priv        CPU Time    Elapsed Time
Idle                  0   0   2    0     60    27:20:10.671    27:46:17.417
System                4   8 124 2544    192     4:25:03.921    27:46:17.417
Registry             92   8   4    0   4844     0:01:36.187    27:47:45.092
smss                324  11   2   53   1056     0:00:00.921    27:46:17.277
csrss               420  13  10  408   1784     0:00:30.796    27:45:59.327
wininit             512  13   1  164   1372     0:00:00.875    27:45:57.781
csrss               520  13  12  635   2152     0:10:49.500    27:45:57.748
winlogon            608  13   4  274   2576     0:00:05.765    27:45:57.342
services            644   9   6  372   3900     0:01:04.203    27:45:57.074
lsass               664   9   9 1494   8812     0:02:09.453    27:45:56.855
svchost             764   8  16 1271  12112     0:04:36.781    27:45:54.991
fontdrvhost         784   8   5   36   3400     0:00:05.171    27:45:54.905
fontdrvhost         792   8   5   36   1288     0:00:00.859    27:45:54.905
svchost             876   8  12 1065   8588     0:10:14.500    27:45:54.127
dwm                 988  13  16 1064  46436     0:14:10.312    27:45:53.092
svchost             368   8  66 2470  98080     1:39:35.265    27:45:51.872
svchost             424   8  20  736  84708     0:30:27.187    27:45:51.488
svchost             720   8  16  814  18280     0:02:06.406    27:45:51.170
svchost             704   8  14  423  15304     0:02:53.031    27:45:51.130
svchost            1160   8  24 1063  12544     0:01:38.187    27:45:49.765
Memory Compression 1404   8  66    0    604     0:11:07.046    27:45:44.728
svchost            1436   8  17  989  12204     0:04:12.750    27:45:44.247
svchost            1508   8   2  227   2328     0:00:04.406    27:45:41.553
svchost            1644   8  10  356   3176     0:00:17.468    27:45:39.140
```

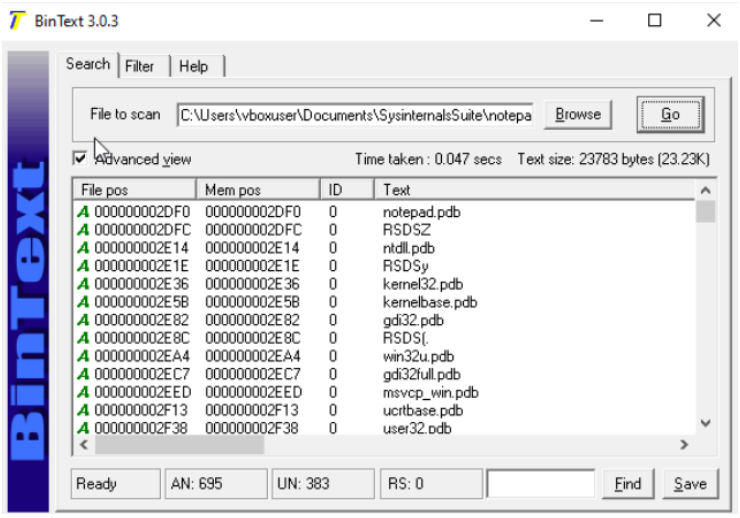
```
cmd                5308   8   1   74   2384     0:00:00.390     0:23:41.548
conhost            7532   8   4  261   8468     0:00:11.234     0:23:41.405
svchost            5348   8   3  163   1684     0:00:00.328     0:17:28.052
svchost            8652   8   4  126   1704     0:00:00.062     0:04:49.122
RuntimeBroker      4364   8   7  209   2840     0:00:00.312     0:00:48.158
smartscreen        6352   8  13  423   8188     0:00:00.515     0:00:20.755
notepad           11980   8   6  237   2972     0:00:00.234     0:00:20.095
pslist             9532  13   3  213   2452     0:00:00.687     0:00:01.494
```

Because the list was very lengthy, I included only the beginning and the end of it.

In order to dump the memory of a specific process, we can use the command **procdump**. I entered the following command to create the memory dump of the Notepad process: **procdump -nobanner -mm 11980**.

```
C:\Users\vboxuser\Documents\SysinternalsSuite>procdump -nobanner -mm 11980
[19:08:51]Dump 1 info: Available space: 19776159744
[19:08:51]Dump 1 initiated: C:\Users\vboxuser\Documents\SysinternalsSuite\notepad
.exe_260225_190851.dmp
[19:08:51]Dump 1 complete: 1 MB written in 0.3 seconds
[19:08:52]Dump count reached.
```

To display the contents of the dump we just created, we used **McAfee's BinText** tool. Since the tool was not available to download from the McAfee website, we used **github.com/mfput/McAfee-Tools** to do so. Once **BinText** was downloaded successfully, I ran the program. Within the program, I opened the memory dump I created in the previous step and selected "GO." This list had four different columns listed as file position, memory position, ID, and text.




When a process is created on a device, that device assigns a unique identifying number to it called the process ID or PID. The next element that is created after beginning a process is the set of handles. The handles can be used to access resources. To display a list of all processes running on a device and their handlers, we can use the command **handle**. This list is extremely lengthy, which is why I only included the beginning of it within the screenshot below.

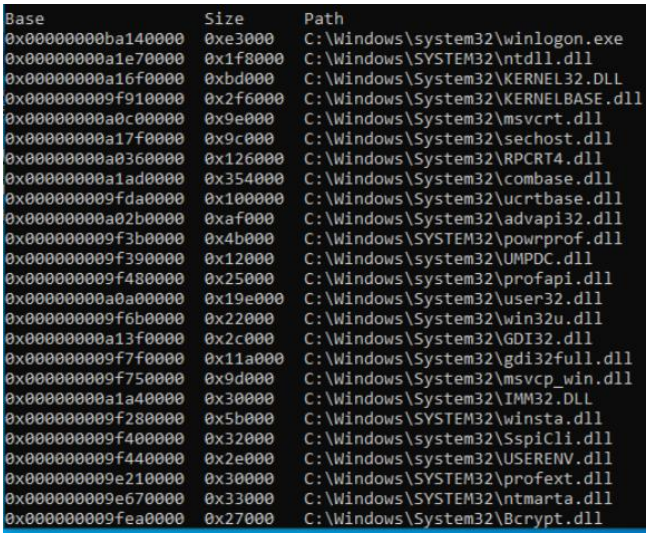
```
C:\Users\vboxuser\Documents\SysinternalsSuite>handle

Nthandle v5.0 - Handle viewer
Copyright (C) 1997-2022 Mark Russinovich
Sysinternals - www.sysinternals.com

------------------------------------------------------------------------------
System pid: 4 \NT AUTHORITY\SYSTEM
------------------------------------------------------------------------------
smss.exe pid: 324 \<unable to open process>
------------------------------------------------------------------------------
csrss.exe pid: 420 \<unable to open process>
------------------------------------------------------------------------------
wininit.exe pid: 512 \<unable to open process>
------------------------------------------------------------------------------
csrss.exe pid: 520 \<unable to open process>
------------------------------------------------------------------------------
winlogon.exe pid: 608 NT AUTHORITY\SYSTEM
   40: File  (RW-)   C:\Windows\System32
  1A8: Section       \Sessions\1\Windows\Theme1038637390
  28C: Section       \Windows\Theme3528996545
  290: File  (R-D)   C:\Windows\System32\en-US\winlogon.exe.mui
  2A8: Section       \Sessions\1\Windows\ThemeSection
  3F0: File  (R-D)   C:\Windows\System32\en-US\user32.dll.mui
------------------------------------------------------------------------------
services.exe pid: 644 \<unable to open process>
------------------------------------------------------------------------------
lsass.exe pid: 664 NT AUTHORITY\SYSTEM
   40: File  (RW-)   C:\Windows\System32
  118: Section       \LsaPerformance
```

If we wanted to display the handles associated with only a specific process, we would use a command like **handle -p 14196**.

```
C:\Users\vboxuser\Documents\SysinternalsSuite>handle -p 14196

Nthandle v5.0 - Handle viewer
Copyright (C) 1997-2022 Mark Russinovich
Sysinternals - www.sysinternals.com

   8C: File  (RW-)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115
   CC: File  (RW-)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115
  19C: File  (RWD)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115\icudtl.dat
  1A4: File  (RW-)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115\v8_context_snapshot.bin
  1AC: File  (R-D)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115\msedge_100_percent.pak
  1B4: File  (R-D)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115\msedge_200_percent.pak
  1BC: File  (R-D)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115\Locales\en-US.pak
  1C4: File  (R-D)   C:\Program Files (x86)\Microsoft\Edge\Application\144.0.3719
.115\resources.pak
  28C: File  (RW-)   C:\Users\vboxuser\AppData\Local\Microsoft\Edge\User Data\Def
ault\Local Storage\leveldb\LOG
  290: File  (RW-)   C:\Users\vboxuser\AppData\Local\Microsoft\Edge\User Data\Def
ault\Local Storage\leveldb\MANIFEST-000004
  298: File  (RW-)   C:\Users\vboxuser\AppData\Local\Microsoft\Edge\User Data\Def
ault\Local Storage\leveldb\000005.ldb
  29C: File  (RW-)   C:\Users\vboxuser\AppData\Local\Microsoft\Edge\User Data\Def
ault\Local Storage\leveldb\000009.log
```

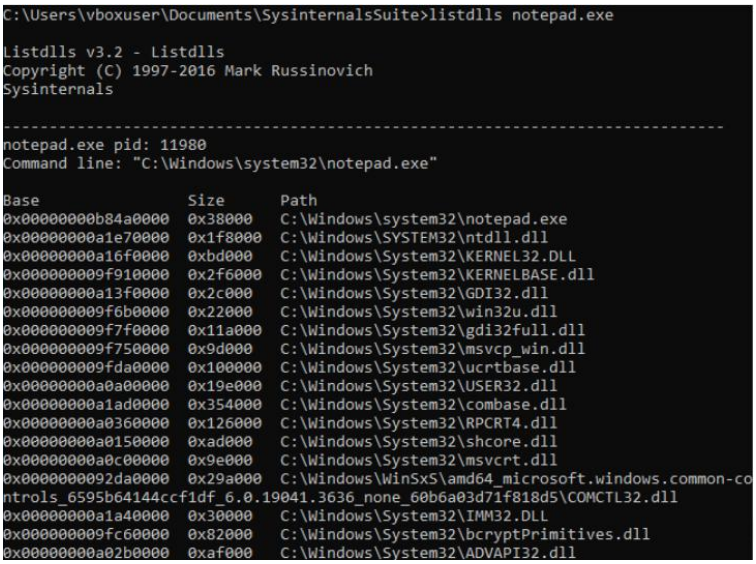
The last few commands we focused on in this part of the lab create detailed lists pertaining to the executable and the dynamic link library (DLL files) that are loaded into processes. The first command we use for this matter is **listdlls**. The output received from entering this command lists all the processes and loaded dynamic link library files that are on the device being used. Due to the length of this list, I only included a small snippet of it in the two screenshots below.

```
C:\Users\vboxuser\Documents\SysinternalsSuite>listdlls

Listdlls v3.2 - Listdlls
Copyright (C) 1997-2016 Mark Russinovich
Sysinternals

Error opening System(4):
Access is denied.

Error opening Registry(92):
Access is denied.

Error opening smss.exe(324):
Access is denied.

Error opening csrss.exe(420):
Access is denied.

Error opening wininit.exe(512):
Access is denied.

Error opening csrss.exe(520):
Access is denied.

------------------------------------------------------------------------------
winlogon.exe pid: 608
Command line: winlogon.exe

Base                Size      Path
0x00000000ba140000  0xe3000   C:\Windows\system32\winlogon.exe
```



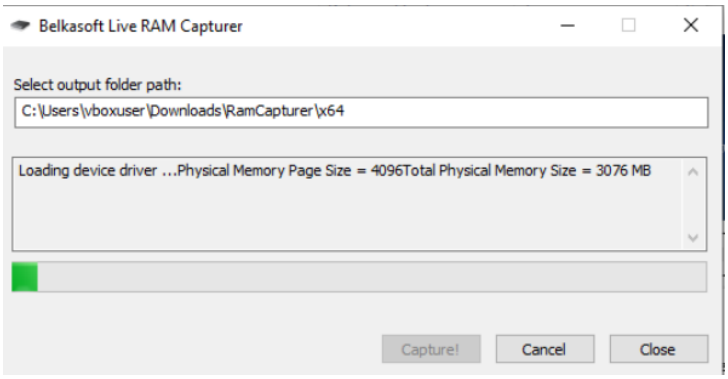
```
Base                Size      Path
0x00000000ba140000  0xe3000   C:\Windows\system32\winlogon.exe
0x00000000a1e70000  0x1f8000  C:\Windows\SYSTEM32\ntdll.dll
0x00000000a16f0000  0xbd000   C:\Windows\System32\KERNEL32.DLL
0x000000009f910000  0x2f6000  C:\Windows\System32\KERNELBASE.dll
0x00000000a0c00000  0x9e000   C:\Windows\System32\msvcrt.dll
0x00000000a17f0000  0x9c000   C:\Windows\System32\sechost.dll
0x00000000a0360000  0x126000  C:\Windows\System32\RPCRT4.dll
0x00000000a1ad0000  0x354000  C:\Windows\System32\combase.dll
0x000000009fda0000  0x100000  C:\Windows\System32\ucrtbase.dll
0x00000000a02b0000  0xaf000   C:\Windows\System32\advapi32.dll
0x000000009f3b0000  0x4b000   C:\Windows\SYSTEM32\powrprof.dll
0x000000009f390000  0x12000   C:\Windows\system32\UMPDC.dll
0x000000009f480000  0x25000   C:\Windows\system32\profapi.dll
0x00000000a0a00000  0x19e000  C:\Windows\System32\user32.dll
0x000000009f6b0000  0x22000   C:\Windows\System32\win32u.dll
0x00000000a13f0000  0x2c000   C:\Windows\System32\GDI32.dll
0x000000009f7f0000  0x11a000  C:\Windows\System32\gdi32full.dll
0x000000009f750000  0x9d000   C:\Windows\System32\msvcp_win.dll
0x00000000a1a40000  0x30000   C:\Windows\System32\IMM32.DLL
0x000000009f280000  0x5b000   C:\Windows\SYSTEM32\winsta.dll
0x000000009f400000  0x32000   C:\Windows\system32\SspiCli.dll
0x000000009f440000  0x2e000   C:\Windows\system32\USERENV.dll
0x000000009e210000  0x30000   C:\Windows\SYSTEM32\profext.dll
0x000000009e670000  0x33000   C:\Windows\SYSTEM32\ntmarta.dll
0x000000009fea0000  0x27000   C:\Windows\System32\Bcrypt.dll
```

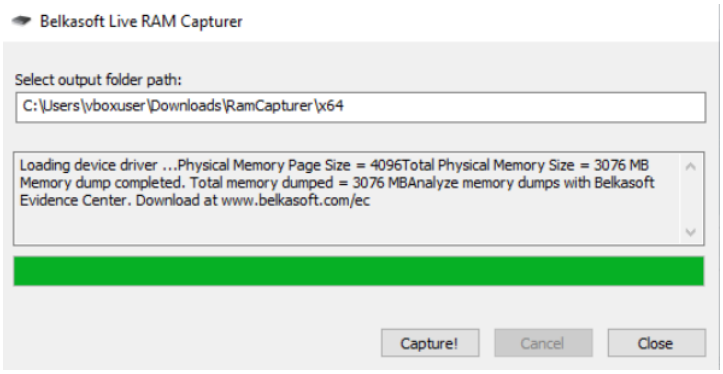
In order to narrow our search a bit and display solely the DLLs associated with a particular process, we used the command **listdlls notepad.exe**.



```
C:\Users\vboxuser\Documents\SysinternalsSuite>listdlls notepad.exe

Listdlls v3.2 - Listdlls
Copyright (C) 1997-2016 Mark Russinovich
Sysinternals

------------------------------------------------------------------------------
notepad.exe pid: 11980
Command line: "C:\Windows\system32\notepad.exe"

Base                Size      Path
0x00000000b84a0000  0x38000   C:\Windows\system32\notepad.exe
0x00000000a1e70000  0x1f8000  C:\Windows\SYSTEM32\ntdll.dll
0x00000000a16f0000  0xbd000   C:\Windows\System32\KERNEL32.DLL
0x000000009f910000  0x2f6000  C:\Windows\System32\KERNELBASE.dll
0x00000000a13f0000  0x2c000   C:\Windows\System32\GDI32.dll
0x000000009f6b0000  0x22000   C:\Windows\System32\win32u.dll
0x000000009f7f0000  0x11a000  C:\Windows\System32\gdi32full.dll
0x000000009f750000  0x9d000   C:\Windows\System32\msvcp_win.dll
0x000000009fda0000  0x100000  C:\Windows\System32\ucrtbase.dll
0x00000000a0a00000  0x19e000  C:\Windows\System32\USER32.dll
0x00000000a1ad0000  0x354000  C:\Windows\System32\combase.dll
0x00000000a0360000  0x126000  C:\Windows\System32\RPCRT4.dll
0x00000000a0150000  0xad000   C:\Windows\System32\shcore.dll
0x00000000a0c00000  0x9e000   C:\Windows\System32\msvcrt.dll
0x0000000092da0000  0x29a000  C:\Windows\WinSxS\amd64_microsoft.windows.common-co
ntrols_6595b64144ccf1df_6.0.19041.3636_none_60b6a03d71f818d5\COMCTL32.dll
0x00000000a1a40000  0x30000   C:\Windows\System32\IMM32.DLL
0x000000009fc60000  0x82000   C:\Windows\System32\bcryptPrimitives.dll
0x00000000a02b0000  0xaf000   C:\Windows\System32\ADVAPI32.dll
```

In the last part of this lab, we concentrated on RAM acquisition. When acquiring RAM, the acquisition must be live. To do so, we have the option to use either the Belkasoft RAM Capturer or the AccessData FTK Imager. I chose to use Belkasoft RAM Capturer and downloaded the software from the Belkasoft website. Once the download was complete, I unzipped the file and ran the application. The Belkasoft Live RAM Capturer window appeared, and I selected the “CAPTURE!” button. After a short while, the memory dump was complete.




## Part 3: References

1. Lab 7: Windows Forensics Part II
2. Lecture notes: Windows Forensics Part 2